

День 1. Бази даних, частина 1

Реляційна модель, ключі та нормалізація

Що таке реляційна база даних

Сьогодні розбираємо першу частину теми «Бази даних». Це один з найважливіших розділів іспиту — на нього припадає від десяти до чотирнадцяти відсотків балів. Гарна новина в тому, що тут майже немає громіздких обчислень. Іспит перевіряє, чи розумієш ти концепції. Тож наша мета — не зазубрити, а зрозуміти логіку. Почнемо з того, що таке реляційна база даних, потім розберемося з ключами, а далі — головне на сьогодні — з нормалізацією.

Уяви звичайну таблицю, як в електронній таблиці: є рядки і є стовпці. У реляційній моделі така таблиця називається відношенням. Кожен рядок — це окремий запис, наприклад один студент або одне замовлення. Кожен стовпець — це атрибут, тобто характеристика: ім'я, дата народження, номер групи. Уся база даних — це набір таких таблиць, які пов'язані між собою. Слово «реляційна» походить від англійського «relation», тобто «відношення». Керує всім цим спеціальна програма — система керування базами даних, англійською Database Management System, скорочено DBMS. Найвідоміші приклади — PostgreSQL, MySQL, Oracle, Microsoft SQL Server.

Ключі

Тепер ключі. Це поняття, яке часто плутають, тож слухай уважно. Ключ — це атрибут або набір атрибутів, який допомагає однозначно знайти потрібний рядок.

Найголовніший — первинний ключ, англійською *primary key*. Це стовпець, значення якого унікальне для кожного рядка й ніколи не порожнє. Наприклад, у таблиці студентів первинним ключем може бути номер залікової книжки. Двох студентів з однаковим номером бути не може, тому за цим номером ми завжди знайдемо рівно одного студента. Первинний ключ у таблиці лише один.

Буває, що однозначно ідентифікувати рядок може не один стовпець, а кілька кандидатів. Скажімо, студента можна однозначно знайти і за номером залікової книжки, і за номером паспорта. Обидва ці варіанти називаються потенційними, або кандидатними ключами. З-поміж усіх кандидатних ключів ми обираємо один і призначаємо його первинним. Решта залишаються просто кандидатними.

Іноді жоден окремий стовпець не є унікальним, і доводиться брати комбінацію кількох. Наприклад, у таблиці оцінок один студент має багато оцінок, і один предмет складає багато студентів. Тому ані стовпець «студент», ані стовпець «предмет» окремо не унікальні. А от їхня пара — конкретний студент плюс конкретний предмет — уже унікальна. Такий ключ із кількох стовпців називається складеним, або композитним. Запам'ятай це слово, воно нам знадобиться, коли говоритимемо про нормалізацію.

І останній тип — зовнішній ключ, англійською *foreign key*. Це стовпець в одній таблиці, який посилається на первинний ключ іншої таблиці. Саме зовнішні ключі зв'язують таблиці між собою. Наприклад, у таблиці оцінок є стовпець «номер залікової книжки», і він посилається на таблицю студентів. Так база розуміє, якому саме студентові належить кожна оцінка. Завдяки зовнішнім ключам ми не дублюємо всю інформацію про студента в кожному рядку оцінок, а просто посилаємося на нього.

Нормалізація: навіщо вона потрібна

Тепер переходимо до головного на сьогодні — нормалізації. Це, мабуть, найчастіше питання з усієї теми баз даних, тож розберемо ґрунтовно.

Уяви, що ми зберігаємо всі дані в одній великій таблиці: в кожному рядку оцінки записані ім'я студента, його адреса, телефон, назва предмета, ім'я викладача і сама оцінка. Що тут погано? По-перше, дані дублюються. Якщо студент склав десять іспитів, його адреса й телефон повторюються десять разів. Це марнування пам'яті. По-друге, і це гірше, виникають так звані аномалії. Якщо студент змінив телефон, нам треба виправити його в усіх десяти рядках, і якщо ми забудемо хоча б один, дані стануть суперечливими. Це аномалія оновлення. Якщо ми хочемо додати нового викладача, який ще нікого не екзаменував, нам нема куди його вписати, бо немає оцінки. Це аномалія вставлення. А якщо ми видалимо останню оцінку з якогось предмета, разом з нею зникне і вся інформація про цей предмет. Це аномалія видалення.

Нормалізація — це процес розбиття однієї великої таблиці на кілька менших і пов'язаних так, щоб усунути дублювання й ці аномалії. Робиться це поетапно, і кожен етап називається нормальною формою. Розберемо їх по черзі: перша, друга, третя і так звана нормальна форма Бойса-Кодда.

Нормальні форми

Перша нормальна форма, скорочено 1NF. Вимога проста: усі значення в таблиці мають бути атомарними, тобто неподільними. В одній клітинці — рівно одне значення, і жодних списків чи повторюваних груп. Наприклад, якщо в стовпці «телефони» записано через кому три номери — це порушення першої нормальної форми. Щоб виправити,

кожен номер треба винести в окремий рядок або окрему таблицю.

Простими словами: одна клітинка — один факт.

Друга нормальна форма, 2NF. Вона має сенс лише тоді, коли первинний ключ складений, тобто складається з кількох стовпців.

Згадай нашу таблицю оцінок, де ключ — пара «студент плюс предмет».

Вимога другої нормальної форми така: кожен неключовий атрибут має залежати від усього складеного ключа, а не від його частини.

Наприклад, ім'я студента залежить тільки від студента, а не від предмета. Отже, воно залежить лише від частини ключа — це порушення. Розв'язання: винести ім'я студента в окрему таблицю студентів, де ключем буде сам студент. Якщо коротко: друга нормальна форма усуває часткові залежності від складеного ключа. Якщо ключ простий, з одного стовпця, то таблиця, що вже в першій нормальній формі, автоматично і в другій.

Третя нормальна форма, 3NF. Вимога: жоден неключовий атрибут не повинен залежати від іншого неключового атрибута. Така залежність називається транзитивною. Поясню на прикладі. Нехай у таблиці студентів є стовпці: номер групи і назва факультету. Номер групи залежить від студента, це нормально. Але назва факультету залежить від номера групи, а не безпосередньо від студента. Тобто маємо ланцюжок: студент визначає групу, група визначає факультет. Це і є транзитивна залежність, і це порушення третьої нормальної форми. Розв'язання: винести зв'язок «група — факультет» в окрему таблицю груп. Якщо коротко: третя нормальна форма усуває залежності одних неключових атрибутів від інших.

Є зручна приказка англійською, яка описує суть першої, другої і третьої форм. Кожен неключовий атрибут має залежати від ключа, від усього ключа і ні від чого, окрім ключа. «Від ключа» — це перша й друга

форми разом, «від усього ключа» — друга форма, «ні від чого, окрім ключа» — третя форма.

І нарешті нормальна форма Бойса-Кодда, скорочено BCNF. Це посилена версія третьої форми. Вимога формулюється так: будь-який детермінант має бути потенційним ключем. Детермінант — це атрибут або набір атрибутів, від якого однозначно залежать інші. Звучить складно, тож запам'ятай головне: BCNF суворіша за третю нормальну форму і вона закриває деякі рідкісні випадки, які третя форма пропускає. Зокрема, ті, де є кілька складених кандидатних ключів, що перетинаються. На більшості іспитових питань достатньо розуміти, що це найсуворіша з чотирьох форм і що кожна наступна форма включає вимоги всіх попередніх. Тобто якщо таблиця в третій нормальній формі, то вона автоматично і в першій, і в другій.

Підсумок

Підсумую сьогоднішнє коротко. Реляційна база — це набір пов'язаних таблиць. Первинний ключ однозначно ідентифікує рядок, зовнішній ключ зв'язує таблиці. Нормалізація розбиває дані, щоб прибрати дублювання й аномалії. Перша форма вимагає атомарних значень. Друга прибирає часткові залежності від складеного ключа. Третя прибирає транзитивні залежності між неключовими атрибутами. Форма Бойса-Кодда — найсуворіша. Кожна наступна форма включає попередні.

На цьому перша лекція завершена. Послухай її ще раз перед сном, якщо є можливість, — повторення на слух дуже добре закріплює саме такі визначення. А далі переходь до тестових питань, щоб перевірити себе.